

// system design

# Os 7 padrões que aparecem em toda entrevista de system design

Os problemas mudam. Os padrões, não.



01 Scaling Reads



02 Scaling Writes



03 Long-Running Tasks



04 Real-time Updates



05 Large Files



06 Contention



07 Multi-step Processes



# Scaling Reads

## o problema

100 leituras pra cada escrita. O banco de dados vira o gargalo, e a primeira reação de todo mundo é adicionar infraestrutura.

post do Instagram: escrito 1x, lido milhões

## a solução

Uma progressão, sempre nessa ordem: índice → connection pool → réplica de leitura → cache → CDN.  
Bancos modernos aguentam mais do que você assume.

na entrevista

Diga ONDE está o gargalo antes de propor camada nova.  
Quem pula direto pra "adiciona um Redis" sem medir nada  
falha aqui.



# Scaling Writes

## o problema

100 milhões de eventos por dia são 1.157 escritas por segundo, em média. Em pico, 10x isso. Cada interação dispara múltiplas escritas.

um Postgres típico aguenta 10-20k escritas/s

## a solução

Quatro estratégias: sharding pra dividir dados, batching pra agrupar, fila pra absorver pico, e o banco certo pro workload.

na entrevista

Faça a conta em voz alta antes de propor sharding.  
Dimensionar primeiro mostra maturidade; particionar sem número mostra decoreba.







# Real-time Updates

## o problema

HTTP é pergunta e resposta: o cliente pede, o servidor responde, a conexão fecha. Mas quem avisa o cliente quando algo muda no servidor?

WhatsApp: a mensagem simplesmente aparece

## a solução

Escada de complexidade: polling (simples, atrasa) → SSE (servidor empurra, uma direção) → WebSocket (duas direções, mais caro de operar).

## na entrevista

Comece propondo polling e justifique cada subida de degrau. Quem abre com WebSocket pra um dashboard de 30s em 30s complicou sem precisar.





# Large Files

## o problema

Upload de 2GB passando pelo seu servidor: CPU, memória e conexões presas só pra mover bytes. E você paga a banda duas vezes.

falhou em 99%? recomeça do zero

## a solução

Presigned URLs: o servidor gera uma permissão temporária e o cliente sobe direto pro S3. Upload em chunks resolve o recomeço do zero.

## na entrevista

A frase que resolve o slide: "meu servidor gerencia permissões, não bytes". Downloads seguem a mesma lógica, na direção oposta.



# Contention

## o problema

Último ingresso. Ana e Bruno clicam comprar no mesmo instante. Os dois leem "1 disponível", os dois pagam. Um fica na porta.

`race condition: ler → (mundo muda) → gravar`

## a solução

Feche o intervalo entre ler e gravar: transação com lock pessimista (trava antes) ou lock otimista (versiona e rejeita o segundo).

## na entrevista

Desenhe a timeline dos dois cliques antes de falar em lock. Quem mostra o intervalo entende o problema; quem decora "usa mutex" não.



# Multi-step Processes

## o problema

E-commerce: cobrar → reservar estoque → gerar etiqueta → confirmar por email. O passo 3 falhou. Cliente pagou e não recebe. Quem desfaz o quê?

`workflow distribuído falha em qualquer passo`

## a solução

Saga: cada passo tem uma compensação planejada (estornar, liberar estoque). Ou workflow engine (Temporal) pra orquestrar retry e rollback.

## na entrevista

Enumere os modos de falha passo a passo antes da solução. A pergunta escondida aqui nunca é o happy path, é o rollback.



```
// o que separa quem passa
```

**Decorar resposta não  
passa. Reconhecer o  
padrão, sim.**

A entrevista nunca vai ser o problema que você estudou.  
Vai ser um problema novo, feito dos mesmos 7 blocos.

o artigo completo, com diagramas

**newsletter.nagringa.dev**

link no primeiro comentário